

네. 지금은 팀원과 **DB 구현 방법 자체보다 “제품 의미”를 먼저 합의해야 하는 시점**입니다. 특히 아래 항목들은 한 명이 임의로 결정하면 나중에 트렌드 기능 전체에 영향을 줍니다.

현재까지 정한 방향 중 **사용자는 활성 대분류만 관심사로 선택**하는 것은 기존 확정안과 일치하고, **trend_category_map**은 트렌드와 카테고리의 다대다 연결 구조입니다.

1. 반드시 팀원과 합의할 것

① 세부분류 목록 자체

우리가 방향은 정했지만 **실제 Taxonomy 내용은 팀 합의가 필요합니다.**

예:

```
게임
├─ 모바일 게임
├─ PC 게임
├─ 콘솔 게임
└─ e스포츠
```

여기서 팀원과 확인할 것은 단순히 이름이 마음에 드는지가 아니라:

- 졸업작품 MVP에서 어느 정도 세분화가 필요한가
- 너무 세밀하거나 너무 넓은 항목은 없는가
- 앞으로 실제 수집할 트렌드를 대부분 분류할 수 있는가
- **기타** 카테고리를 둘 것인가
- 대분류마다 세부분류 개수를 비슷하게 맞추는 필요가 있는가

입니다.

제가 원하는 기준은 “현재 수집하려는 트렌드를 분류할 만큼만”입니다.

처음부터:

```
게임 20개
패션 30개
음식 25개
```

처럼 만들 필요는 없습니다.

② 하나의 Trend에 여러 세부분류를 허용할 것인가

이건 중요한 제품 규칙입니다.

예를 들어:

갤럭시 AI 기능

은:

AI
모바일/스마트폰

두 세부분류 모두 자연스럽습니다.

현재 `trend_category_map` 자체는 한 Trend에 여러 Category를 연결할 수 있고 대표 Category 하나를 허용하도록 설계되어 있습니다.

따라서 팀원과:

하나의 트렌드가 여러 세부분류에 걸칠 수 있는가?

를 확정해야 합니다.

저는 **YES**를 권합니다.

그러면:

갤럭시 AI 기능

primary
→ 모바일/스마트폰

additional
→ AI

처럼 표현할 수 있습니다.

③ B안의 추천 의미를 확정

이미 이 채팅에서는 B안을 선택했습니다.

Trend
→ 가장 구체적인 세부분류만 저장

사용자 관심사
→ 대분류

추천
→ 대분류의 하위 세부분류까지 포함하여 조회

팀원에게도 이 규칙을 공유하고 동의를 얻는 것이 좋습니다.

예:

```
사용자 관심사
게임

Trend
→ 모바일 게임

결과
→ 게임 사용자의 추천 대상
```

특히 이 결정은 나중에 `GET /api/trends/recommended` 구현에 직접 영향을 줍니다.

현재 오래된 Notion 문서는 사용자 관심 `category_id`와 `trend_category_map.category_id`를 직접 연결하는 방식으로 설명하고 있기 때문에, B안을 채택하면 이 부분은 후에 새 설계에 맞춰 갱신해야 합니다.

2. `category_name` UNIQUE 변경도 팀 합의가 좋습니다

현재 Frozen Schema에는:

```
categories.category_name UNIQUE
```

가 명시되어 있습니다.

우리는 지금:

```
음식
└─ 기타

게임
└─ 기타
```

를 허용하려고 합니다.

따라서 팀에게 다음 식으로 설명하면 됩니다.

"카테고리가 평면 구조였을 때는 이름 전역 UNIQUE가 자연스러웠는데, 이제 부모가 있는 계층 구조를 실제로 사용하므로 서로 다른 부모 아래 같은 이름을 허용할 필요가 있다."

그리고 의논할 핵심은:

```
같은 부모 아래 동일 이름
→ 금지
```

다른 부모 아래 동일 이름
→ 허용

에 동의하는가입니다.

어떤 **SQL Constraint**로 구현할지는 팀 회의에서 굳이 정할 필요 없습니다.

그건 그 다음 기술 설계 단계에서 MariaDB의 **NULL + UNIQUE** 동작까지 고려해서 정하면 됩니다.

3. Trend Attribute는 가장 중요한 미확정 사항입니다

여기는 반드시 팀원과 제품 관점에서 이야기해보는 것을 추천합니다.

예를 들어:

신제품
업데이트
이벤트
میم
논란
콜라보
출시

를 DB에 넣을 것인가 하는 문제입니다.

팀원에게 SQL이나 ENUM 이야기를 먼저 하지 말고 이렇게 물어보세요.

"우리 앱에서 사용자가 '업데이트 트렌드만 보기', 'میم 트렌드만 보기'처럼 이걸 실제로 구분해서 쓸 일이 있는가?"

이 질문이 가장 중요합니다.

팀 답변이 "아니다"

그렇다면 MVP에서는 굳이 구조화된 Attribute가 필요 없습니다.

title
summary
AI related keywords
hashtags

정도로도 충분합니다.

즉 Trend Attribute는 **후속 기능으로 연기**할 수 있습니다.

팀 답변이 "필요하다"

그다음 질문입니다.

"하나의 트렌드가 여러 유형을 동시에 가질 수 있는가?"

예:

게임 x 애니메이션 콜라보 이벤트

COLLABORATION
EVENT

화장품 신제품 논란

NEW_PRODUCT
CONTROVERSY

여기에 팀도 "그럴 수 있다"고 본다면 단일:

trend_type ENUM(...)

은 좋은 모델이 아닐 가능성이 높습니다.

그때 M:N Attribute 구조를 검토하면 됩니다.

4. 해시태그는 딱 한 가지만 합의하면 됩니다

이번에 정한 잠정 규칙은 꽤 명확합니다.

현재 Frozen Schema의 `trend_related_keywords`에는 `RELATED` / `HASHTAG` / `RECOMMENDED` 유형이 있고, AI Analysis에 연결됩니다.

따라서 팀과 확인할 것은:

MVP의 해시태그는 AI가 생성/추천하는 정보인가, 실제 SNS에서 수집할 데이터인가?

입니다.

MVP에서는 AI 추천용이다

현재 구조 그대로 갑니다.

```
trend_ai_analyses
  ↓
trend_related_keywords
  ↓
HASHTAG
```

실제 SNS 해시태그를 수집할 예정이다

이건 별도 설계 작업으로 빼는 것이 좋습니다.

```
trend_source_hashtags
→ trend_sources
```

같은 구조를 나중에 검토합니다.

저는 현재 MVP에서는 AI 생성/추천 Hashtag만 두고 실제 Source Hashtag 저장은 연기하는 쪽을 권합니다.

5. 기타를 돌지 의외로 꼭 논의해보세요

category_name UNIQUE 문제를 발견하게 만든 예시이기도 하지만, 제품적으로도 의미가 있습니다.

두 방식이 있습니다.

기타 있음

```
음식
├─ 한식
├─ 양식
├─ 중식
├─ 일식
├─ 카페/디저트
└─ 기타
```

새로운 Trend를 어떻게든 분류할 수 있습니다.

하지만 많은 Trend가:

```
기타
기타
기타
```

로 들어가면 Taxonomy 가치가 떨어집니다.

기타 없음

기존 분류로 표현하기 어려운 Trend를 발견하면:

분류 실패
→ Taxonomy 개선 후보

로 볼 수 있습니다.

프로토타입 단계에서는 오히려 이게 설계 개선에 도움이 됩니다.

개인적으로는 초기 **Seed**에서는 **기타**를 굳이 넣지 않는 방식도 상당히 괜찮다고 봅니다.

실제 데이터를 넣어보고 분류 불가능한 케이스가 반복될 때 추가해도 늦지 않습니다.

6. 팀 회의에서 굳이 논의하지 않아도 되는 것

반대로 이런 것은 구현자가 기술 설계에서 처리해도 됩니다.

Alembic Migration 파일명
UniqueConstraint 이름
Repository 메서드명
JOIN을 어떤 SQLAlchemy 문법으로 작성할지
Pydantic Schema 이름
Seed INSERT SQL 작성 방식

이런 것까지 팀 회의에 올리면 제품 의사결정과 구현 세부사항이 섞입니다.

팀원이 함께 확정해야 하는 것은 *****어떤 상태를 허용할 것인가*****이고,

개발자가 설계할 것은 *****그 규칙을 DB와 코드에서 어떻게 보장할 것인가*****입니다.

7. 실제로 팀원에게 가져갈 회의 안건

이 정도면 충분합니다.

Trend Leader 카테고리/트렌드 분류 논의

- 세부분류는 "주제 영역"으로만 구성한다.
예: 모바일 게임, PC 게임, 한식, 스킨케어
→ 동의 여부
- 실제 6개 대분류별 세부분류 목록을 어디까지 둘 것인가?

3. 하나의 Trend에 여러 세부분류 연결을 허용할 것인가?
제안: 허용, 대표 카테고리 1개
4. Trend에는 가장 구체적인 세부분류만 저장한다.
사용자 대분류 추천 시 자식 카테고리까지 포함해 조회한다.
→ 동의 여부
5. 서로 다른 부모 아래 동일한 category_name을 허용할 것인가?
예: 게임/기타, 음식/기타
제안: 허용
6. 초기 버전에 "기타" 세부분류가 필요한가?
7. 신제품/업데이트/이벤트/밈/논란/콜라보 같은 Trend Attribute를
실제 필터·추천·통계 기능에서 사용할 것인가?
8. 하나의 Trend가 여러 Attribute를 동시에 가질 수 있다고 볼 것인가?
9. MVP 해시태그의 의미는 무엇인가?
제안: 우선 AI 생성/추천 해시태그만 지원
실제 Source Hashtag 수집은 별도 후속 설계

회의가 끝났을 때 필요한 결과

전부 완벽하게 설계할 필요는 없습니다.

딱 다음 정도만 얻으면 성공입니다.

[확정]

- 세부분류 기준
- 6개 대분류별 초기 세부분류 목록
- Trend ↔ Category 다중 연결 여부
- B안 추천 규칙
- 동일 이름 허용 정책
- 기타 사용 여부
- MVP Hashtag 의미

[미확정 가능]

- Trend Attribute 필요 여부
- Attribute 종류
- Attribute DB 구조

특히 **Trend Attribute**는 이번 회의에서도 "아직 필요성을 모르겠다"가 결론이어도 괜찮습니다. 오히려 근거 없이 테이블을 만드는 것보다 훨씬 나은 결정입니다.

팀원과 이야기할 때 가장 중요한 질문 하나만 꼽으면 **"이 데이터를 앱에서 실제로 어디에 사용할 건가?"**입니다. 이 질문으로 Category인지, Attribute인지, Keyword인지, 단순 표시용 정보인지 상당 부분 구분할 수 있습니다.